

Clase 133 — Análisis estático de binarios

Parte: 5 — Explotación de sistemas y binarios · Fuente: *Andriesse, Practical Binary Analysis* 🕒

Duración estimada: 120 min · Nivel: Avanzado

Objetivo

Profundizar en el **análisis estático**: entender un binario sin ejecutarlo. Estudiarás el desensamblado (lineal vs recursivo), la reconstrucción del grafo de control de flujo (CFG) y de llamadas (call graph), el análisis de flujo de datos elemental y los límites del enfoque estático frente a código ofuscado o generado dinámicamente. Es la base rigurosa del reversing profesional.

⚠ **Ética:** solo binarios propios/autorizados.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Distinguir** desensamblado lineal y recursivo y sus fallos típicos.
2. **Reconstruir** el CFG y el call graph de una función.
3. **Aplicar** análisis de flujo de datos básico (def-use, propagación de constantes).
4. **Reconocer** los límites del análisis estático (packing, JIT, indirect calls).
5. **Combinar** herramientas (objdump, Ghidra, capstone) para un análisis reproducible.

Temas

#	Tema	Por qué importa
1	Desensamblado lineal	Simple pero se confunde con datos
2	Desensamblado recursivo	Sigue el flujo; más preciso
3	CFG (control flow graph)	Estructura lógica de la función
4	Call graph	Relaciones entre funciones
5	Data flow básico	Rastrear valores y constantes
6	Detección de funciones	Prólogos y heurísticas
7	Límites: packing, indirección	Cuándo el estático no basta
8	capstone/pyelftools	Automatizar el análisis

Definiciones y características

- **Desensamblado lineal:** decodifica byte a byte de inicio a fin. *Clave:* rápido, pero interpreta datos incrustados como código (errores de sincronización).
- **Desensamblado recursivo:** sigue saltos/llamadas desde el entry point. *Clave:* más preciso; puede perder código alcanzado solo por saltos indirectos.
- **CFG:** grafo de bloques básicos y aristas de salto. *Clave:* revela bucles, ramas y estructura.
- **Call graph:** grafo de qué función llama a cuál. *Clave:* ayuda a priorizar funciones interesantes.
- **Análisis def-use:** dónde se define y usa un valor. *Clave:* base para propagación de constantes y detección de datos controlados por el usuario.
- **Límites del estático:** packing, cifrado, self-modifying code, saltos indirectos. *Clave:* exigen complementar con análisis dinámico (clase 134).

Herramientas y preparación

```
pip install capstone pyelftools
sudo apt install -y binutils
# Ghidra para CFG/decompilado (clase 131)
```

Laboratorio guiado

Entorno propio.

1. Compara desensamblado lineal vs recursivo en un binario con datos incrustados:

```
bash objdump -d crackme # lineal (GNU) # En Ghidra/IDA/r2 el análisis es recursivo: contrasta
la función donde difieren
```

2. Reconstruye el CFG de `main` en Ghidra (Window → Function Graph) e identifica bucles y ramas de éxito/fracaso.

3. Escribe un desensamblador mínimo con capstone para ver cómo se decodifican instrucciones:

```
python from capstone import * from elftools.elf.elffile import ELFFile f =
ELFFile(open("crackme", "rb")) text = f.get_section_by_name(".text") code, addr = text.data(),
text["sh_addr"] md = Cs(CS_ARCH_X86, CS_MODE_64) for i in md.disasm(code, addr):
print(f"0x{i.address:x}:\t{i.mnemonic}\t{i.op_str}")
```

4. Haz un análisis def-use manual de la variable de entrada: dónde se lee (`scanf`), dónde se compara, qué transformación sufre.
5. Empaqueta un binario con `upx` y muestra que el desensamblado estático solo ve el stub (límite del estático):

```
bash upx -9 crackme -o crackme.packed objdump -d crackme.packed | head # apenas el
descompresor
```

6. Documenta qué preguntas quedan sin responder por estático y requerirán dinámico.

Ejercicios

1. Encuentra un caso donde el desensamblado lineal se desincronice.
2. Dibuja el CFG de una función con un bucle y una condición.
3. Construye el call graph parcial de un binario pequeño.
4. Usa capstone para contar cuántas instrucciones `call` hay en `.text`.
5. Detecta con `upx -t` /entropía si un binario está empacado.
6. Identifica un salto indirecto (`jmp rax`) y explica por qué complica el estático.

Reto verificable

Con capstone y pyelftools, escribe un script que liste todas las instrucciones `call` de `.text` con su dirección y, cuando sea directo, la función destino.

Criterio de aceptación: el script imprime las llamadas directas resueltas correctamente (verificable contra `objdump -d`) y marca las indirectas como "no resueltas".

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Desensamblado "basura" a mitad	Datos incrustados; usa análisis recursivo (Ghidra/r2)
Faltan funciones en el CFG	Alcanzadas por saltos indirectos; complementa con dinámico
capstone no decodifica	Modo/arquitectura equivocados (32 vs 64)
Binario "vacío" en objdump	Está empacado (UPX u otro); desempácalo primero
Call destino incorrecto	Confundes dirección relativa con absoluta

Preguntas frecuentes

 **¿Estático o dinámico?** El estático da visión global sin ejecutar; el dinámico revela lo que solo ocurre en runtime. Se complementan.

 **¿Cómo detecto packing?** Alta entropía, pocas secciones, imports mínimos, o firmas conocidas (`upx`).

 **¿Puedo resolver saltos indirectos estáticamente?** A veces con análisis de valores; en general requieren ejecución/emulación.

Referencias

- Andriesse, D. *Practical Binary Analysis*, caps. 5-8. No Starch Press.
- Capstone Engine — <https://www.capstone-engine.org/>
- pyelftools — <https://github.com/eliben/pyelftools>
- UPX — <https://upx.github.io/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.

-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiente clase

[Clase 134 - Analisis dinamico y debugging de binarios](#)